

Programming Language Principles, Design, and Implementation

Lecture 1 - Introduction

Sean Moss

University of Birmingham

Where are we?

- ▶ **Part 1: Principles of programming**
 - ▶ **Functional languages**
 - ▶ Operational semantics
 - ▶ Type systems
 - ▶ Abstract datatypes
- ▶ Part 2: Compilers

Today

- ▶ Functional programming: what and why?
- ▶ Organization of the first 5 weeks
- ▶ Programming in Haskell

Further reading:

- ▶ “A Gentle Introduction to Haskell”
<https://www.haskell.org/tutorial/>
- ▶ Almost anything you want to know about Haskell can be found here:
<https://www.haskell.org>
- ▶ UPenn CIS 194 lectures:
<https://www.seas.upenn.edu/~cis1940/spring13/>
- ▶ Online Haskell programming: <http://code.world/haskell>

Functional Programming

We often use the word **functional** to refer to programming languages where

- ▶ functions are **first-class**, i.e., they can be used the same ways as other kinds of value (e.g., numbers)
functions can be passed as arguments to other functions
- ▶ programs evaluate expressions rather than execute instructions
functions do not modify their arguments.

Example in Haskell:

```
1 plus1 x = x + 1
2 plus2 x = x + 2
3 myadd x g h = g x + h x
4 main = print (myadd 2 plus1 plus2)
```

Prints: 7

The functions `plus1` and `plus2` are passed as arguments to `add`

Functional Programming

Several key properties are often used to distinguish between classes of functional programming languages:

Pure: often refers to programming languages where programs cannot have side effects

- ▶ pure: Haskell (its core)
- ▶ impure: OCaml
- ▶ benefits: easier to handle concurrency, debug, maintain, reason about

Functional Programming

Lazy/eager: different programming languages rely on different evaluation strategies. Lazy means that expressions are not evaluated until they are needed (eager means that they are).

- ▶ lazy: Haskell
- ▶ eager: OCaml
- ▶ **will be discussed Week 3**

This terminates in Haskell:

```
1 unit f = putStrLn "done"  
2 loop () = loop ()  
3 main = unit (loop ())
```

but not in OCaml: Why?

```
1 let unit f = print_endline "done"  
2 let rec loop () = loop ()  
3 let main = unit (loop ())
```

Functional Programming

Static typing: expressions have types that are checked at compile-time. Programs with type errors are rejected at compile-time.

- ▶ static typing: Haskell, OCaml, etc.
- ▶ dynamic typing: Lisp
- ▶ **will be discussed in Week 4**

Accepted by Haskell: (it can even infer that type at compile-time)

```
1 add1 :: Integer -> Integer
2 add1 x = x + 1
```

Rejected by Haskell at compile-time: Why is it rejected?

```
1 addT x = x + True
```

Why Functional Programming

Often very rich languages that are “fairly” well-defined and convenient to study programming principles.

Simple: the very nature of functional programming leads to simple compact programs.

Reasoning: generally more amenable to verification (especially for pure languages).

In the real world:

- ▶ OCaml: <https://ocaml.org/industrial-users>
- ▶ Haskell: http://wiki.haskell.org/Haskell_in_industry

“Functional” features adopted in other languages:

- ▶ Higher-order functions
- ▶ Type classes
- ▶ etc.

Focus of the first 5 weeks

Functional programming (Week 1)

- ▶ Haskell: the real-world language we use to illustrate these principles

λ -calculus (Week 2)

- ▶ the λ -calculus: a simple programming language.
- ▶ our base language to discuss programming language principles

Evaluation (Week 3)

- ▶ operational semantics of languages: how to compute
- ▶ evaluation order: order in which programs compute

Type systems (Week 4)

- ▶ introduction to type systems
- ▶ type checking & type inference
- ▶ expressiveness: polymorphism

Abstraction (Week 5)

- ▶ data abstraction & information hiding (existential types)
- ▶ abstract data types (modules & type classes)

Organization of the first 5 weeks

Pre-requisites:

- ▶ some exposure to functional programming

Lectures

- ▶ 2 or 3 lectures every week

Assignments

- ▶ 1 formative assignment
- ▶ 1 summative assignment

Resources

- ▶ Canvas page: <https://canvas.bham.ac.uk/courses/77868>
- ▶ Useful textbook: Benjamin C. Pierce, “Types and Programming Languages”

Learning objectives

- ▶ Understand operational semantics
- ▶ Apply typing algorithms to check or infer types

Our First Haskell Program

Haskell

- ▶ a “mostly pure” lazy functional programming language
- ▶ we will use it to illustrate concepts taught in this module
- ▶ it will allow you to experience these concepts
- ▶ but it is not a requirement

Either:

- ▶ **online:** <http://code.world/haskell>
- ▶ **locally:** install GHC

Example in Haskell:

```
1 main = print (2 + 1)
```

Or just **print (2+1)** in GHCi.

Our First Haskell Program

Example in Haskell:

```
1 main = print (2 + 1)
```

- ▶ **online:** <http://code.world/haskell>
Demo
- ▶ **compile:**
 - ▶ store in a file: `test.hs`
 - ▶ compile the code: `ghc -o test test.hs`
 - ▶ execute it: `./test`

Demo

- ▶ **interactive environment:**
 - ▶ store in a file: `test.hs`
 - ▶ execute: `ghci`
 - ▶ load the file: `:l test.hs`
 - ▶ execute the main function: `main`

Demo

Try one (or more) of the above

Some Basic Key Concepts

We will now go through some basic key concepts:

- ▶ Variables
- ▶ Types
- ▶ Functions
- ▶ Higher-Order Functions
- ▶ Datatypes
- ▶ Pattern-matching
- ▶ Recursion

Variables and Numbers

We can declare variables:

```
1 someint = 7
```

- ▶ someint is not mutable. ('Variables do not vary').
- ▶ someint is a name bound to the value 7.

We can make local declarations with `let`:

```
1 let someint = 7 in (let someint = 9 in 2 * someint) + someint
```

A Haskell program must have a “main” function as a top-level definition:

- ▶ use for example: `main = print someint`
- ▶ not necessary when using `ghci`

Variables, Numbers and Types

We can specify the type of definitions:

```
1 someint :: Int
2 someint = 7
```

- ▶ **Int**: machine-sized integers
- ▶ **Integer**: arbitrary size

Other kinds of numbers – double-precision floating point numbers:

```
1 somedouble :: Double
2 somedouble = 7.3
```

Variables, Numbers and Types

We can specify the type of definitions:

```
1 someint :: Int
2 someint = 7
```

Given a definition of type **Int**, what do we know about this definition?

What about?

```
1 someotherint :: Int
2 someotherint = 7 + someotherint
```

Haskell accepts this code because

- ▶ 7 is an **Int**
- ▶ and someotherint is also declared to be an **Int**

Types guarantee that, if an expression returns a value, then it is a value of the right kind. In Haskell they do not guarantee termination.

Functions

We can manipulate data by applying functions to that data:

```
1 increment :: Int -> Int
2 increment x = x + 1
3
4 inc7 :: Int
5 inc7 = increment 7
```

- ▶ increment is a function from **Int** to **Int** as indicated by $\rightarrow$
- ▶ it takes an x and returns $x+1$
- ▶ we don't need parentheses to apply increment to 7

Use parentheses if arguments are more complex:

```
1 inc7m2 :: Int
2 inc7m2 = increment (7 * 2)
```

Alternatively: $\text{inc7m2} = \text{increment } \$ 7 * 2$.

Currying

Juxtaposition denotes application which associates to the left:

$$f\ g\ h = (f\ g)\ h$$

The function-arrow type constructor associates to the right:

$$a \rightarrow b \rightarrow c = a \rightarrow (b \rightarrow c)$$

So a 'two-argument function' is a function that returns another function.

$$x + y = (+)\ x\ y = ((+)\ x)\ y$$

So there are several ways we could define increment.

Use the dollar \$ to work around the associativity of application without brackets

```
1 main = print $ 1 + 2 -- same as print (1 + 2)
```

Higher-Order Functions

We can even apply functions to functions:

```
1 higheradd :: (Int -> Int) -> (Int -> Int) -> Int -> Int
2 higheradd f g x = f x + g x
3
4 someadd :: Int
5 someadd = higheradd ((+) 1) ((+) 2) 3
```

- ▶ higheradd takes two other functions as arguments
- ▶ $(+) 1$ is a **partial application** of $(+)$.
 - ▶ It is equivalent to writing the anonymous function $\backslash x \rightarrow x + 1$.
 - ▶ Haskell provides $(+1)$, $(+2)$, etc. as a shorthand in this case.

What do you think someadd computes to? 9

Datatypes, Pattern-Matching & Recursion

A simple data type:

```
1 data IntList = Nil | Cons Int IntList
```

- ▶ A new type
- ▶ Nil is the empty list
- ▶ (Cons i l) is a list of composed of
 - ▶ a head: the int i
 - ▶ a tail: the list l

Example:

```
1 somelist = Cons 2 (Cons 1 Nil)
```

Datatypes, Pattern-Matching & Recursion

A simple data type:

```
1 data IntList = Nil | Cons Int IntList
```

A **recursive** function is a function that calls itself.

Example:

```
1 length :: IntList -> Integer
2 length Nil = 0
3 length (Cons _ l) = 1 + length l
```

- ▶ length **pattern-matches** on its argument
- ▶ length is called inside the Cons branch

Built-in lists

Something like:

```
1 data [a] = [] | (:) a [a]
```

Haskell provides convenient syntactic sugar:

$$\begin{aligned}[2, 3, 5, 7] &= 2 : 3 : 5 : 7 : [] \\ &= 2 : (3 : (5 : (7 : []))) \\ &= (:) 2 ((:) 3 ((:) 5 ((:) 7 [])))\end{aligned}$$

is the native version of

```
1 Cons 2 (Cons 3 (Cons 5 (Cons 7 Nil)))
```

The λ -calculus

In addition to Haskell, we will consider another **very simple** programming language: the **λ -calculus**

- ▶ A model of computation
- ▶ Same expressive power as Turing machines
- ▶ Can also be seen as a simple programming language
- ▶ We will use it to introduce principles of programming languages
- ▶ Haskell will be used to illustrate some of those points

Only 3 program constructs:

$$M ::= x \mid \lambda x.M \mid M M$$

- ▶ allows creating (anonymous) functions
- ▶ and applying functions

Conclusion

What did we cover today?

- ▶ Functional programming: what and why?
- ▶ Organization of the first 5 weeks
- ▶ Programming in Haskell

Next time?

- ▶ More Haskell programming
- ▶ Using Haskell's datatypes to model a very simple programming language.